

Portfolio Case Studies & Voice Card

1. Voice Card

Direct, plain, precise, technical, warm, no buzzwords.

2. Case Study: Multi-Agent Autonomous Developer

The Problem

AI coding assistants are great at single-turn generation, but complex features usually fail on the first try. Standard single-prompt LLMs lack an execution environment to run code, catch bugs, or verify their own work. I wanted to build a fine-tuned, dedicated system designed specifically to handle the full code-test-repair loop automatically.

What I Did & Decided

I built a multi-agent developer system using LangGraph and Docker that automates writing code, generating tests, and refactoring bugs in a closed loop.

- **Separation of Concerns:** I assigned specialized roles to distinct nodes: a code generator, a Pytest generator, a sandboxed test runner, and a debugger agent.
- **Deterministic Guardrails:** To prevent infinite loops and runaway API costs, I placed a hard cap at 5 debug attempts in the orchestration layer rather than relying on the LLM to decide when to stop.
- **Sandboxed Execution:** All generated code and tests run inside an isolated Docker container with disabled networking (`--network none`) for security.

What Came of It

For standard algorithmic tasks and API boilerplate, the system successfully generated working code and passing test suites in 1 to 2 attempts. The debugger reliably patched syntax errors and off-by-one bugs on the first try.

When testing edge cases, the system hit the 5-attempt limit due to three specific design bottlenecks:

1. **Dependency Isolation:** Without internet access in the container, imports for uninstalled libraries like pandas caused unrecoverable import errors.
2. **The QA Trap:** Flawed assertions generated by the test agent forced the debugger to warp perfectly valid solution code to satisfy broken tests.
3. **Vector Store Poisoning:** Host-level Docker crash logs occasionally got stored in the vector database as successful fixes, injecting non-code text into subsequent runs until memory was wiped.

What I'd Do Differently

Next time, I'd add a **Test Critique Agent** to validate the generated Pytest suite before execution, ensuring the tests themselves are logically sound before the debugger tries to satisfy them.

3. Before / After Comparison

Generic AI Output: "I leveraged state-of-the-art LLM architectures to create a revolutionary, synergistic autonomous software developer that seamlessly optimizes code quality."

Edited Version: "I built a multi-agent developer system using LangGraph and Docker that automates writing code, generating tests, and refactoring bugs in a closed loop."

4. Bio & Contact Prompt

Bio

I build AI systems and machine learning pipelines that solve specific technical problems. Currently studying Chemical Engineering at IIT Kharagpur with a focus on computer vision and multi-agent systems.

Contact Prompt

Have a complex ML or agentic software problem? [NIKHIL MAHINDRAKAR](#)